HALEHOUND DUAL-CORE RF ARCHITECTURE

Eliminating Display Overhead from Radio Operations Using FreeRTOS Core Pinning

Author:	Jesse C. Hale — HaleHound
Platform:	HaleHound-CYD (ESP32-2432S028 + 2x PCF8574 I2C Expanders)
Radios:	3x NRF24L01+PA+LNA + CC1101 + NEO-6M GPS
Date:	February 2026
License:	MIT — github.com/JesseCHale/HaleHound-CYD

99% Radio Duty Cycle

Single ESP32 ¦ Zero Bus Contention ¦ 13 Production Modules

For Authorized Security Research & Educational Purposes Only

Abstract

Every major ESP32-based RF security tool currently runs radio operations and display rendering on the same core. This creates a fundamental performance bottleneck: during each display redraw cycle (~100ms), the radio goes completely silent. The result is a mere 44% radio duty cycle — the tool is inactive more than half the time.

The commercial solution has been to solder a second ESP32 onto the board, doubling hardware cost and complexity. This document describes a software-only architecture that achieves ~99% radio duty cycle on a single ESP32 by leveraging the dual-core Xtensa LX6 processor, FreeRTOS core pinning, and the ESP32's independent HSPI/VSPI hardware buses. This architecture is deployed across thirteen production modules on the HaleHound-CYD platform, covering WiFi, BLE, NRF24 2.4GHz, CC1101 Sub-GHz, and DSP/FFT workloads.

The Problem

In a typical single-core Arduino loop on ESP32 RF tools, execution follows a strict sequential pattern: perform radio work, stop, redraw the display, return to radio. This cycle repeats continuously.

Phase	Duration	Activity
Radio Burst	~80ms	NRF24 channel hopping / TX
Display Draw	~100ms	TFT render + touch polling
Total Cycle	~180ms	Radio duty cycle: only 44%

A 100ms gap is an eternity in RF. BLE advertising intervals are 20–100ms, meaning a target device can complete an entire reconnection handshake while the tool is busy drawing pixels. At 44% duty cycle, the radio is silent more than it is active.

Common workarounds — slowing the display, millis() non-blocking patterns, removing the display entirely, or adding a second ESP32 — all carry severe trade-offs in usability, effectiveness, or cost.

Why Existing Tools Don't Do This

Tutorials don't go far enough — Every dual-core tutorial demonstrates blinking two LEDs on different cores. The jump to running continuous-wave RF carriers with SPI-attached hardware is enormous and undocumented.

A silent SPI bug kills attempts — If SPI is initialized on Core 1 (Arduino default) and used from Core 0, everything appears to work but radio output is weak and malformed. Most developers conclude "dual-core SPI doesn't work."

Firmware-flash culture — The Flipper/Marauder community primarily downloads and flashes .bin files. Developers working within existing frameworks inherit single-threaded architectures.

The CYD's SPI layout is underappreciated — The ESP32-2432S028 routes its TFT and radio SPI to completely independent hardware peripherals (HSPI vs VSPI), enabling zero bus contention without any software

arbitration.

Hardware Prerequisites

This architecture requires two things: a dual-core ESP32 (WROOM-32 or WROVER — not the single-core S2 or C3), and two independent SPI peripherals on separate pin groups.

SPI Bus	Peripheral	Pins
HSPI	TFT Display	GPIO 12 (MISO), 13 (MOSI), 14 (CLK), 15 (CS)
VSPI	NRF24 Radio	GPIO 18 (SCK), 19 (MISO), 23 (MOSI), 4 (CSN)

Zero pin overlap. Two independent hardware buses. No mutex needed.

The Architecture

The design principle is complete hardware isolation between cores. Core 0 exclusively owns the VSPI bus and all radio hardware. Core 1 exclusively owns the HSPI bus, TFT display, and touch input. Neither core ever touches the other's hardware. Cross-core communication uses only volatile variables in shared memory — no mutexes, no queues, no semaphores.

	Core 0 (Radio)	Core 1 (Display)
SPI Bus	VSPI (exclusive)	HSPI (exclusive)
Hardware	NRF24, CC1101, GPS	TFT, Touch, SD Card
Task Type	FreeRTOS pinned task	Arduino loop()
Role	Radio TX/RX, DSP/FFT	UI rendering, touch input
Communication	Writes volatile state	Reads volatile state

The Discovery: SPI ISR Core Affinity

> **This is the single most critical finding in this entire architecture, and the reason most developers fail when attempting dual-core SPI on the ESP32.**

During development of the HaleHound-CYD, the initial dual-core implementation appeared to work perfectly — code compiled, ran without errors, and serial output showed correct operation. However, spectrum analyzer testing revealed the radio output was weak and inconsistent, not the clean continuous carrier expected from an NRF24+PA+LNA at maximum power.

The root cause: the ESP32's SPI driver registers its interrupt service routine (ISR) on whichever core calls spi_bus_initialize(). Arduino's SPI.begin() in setup() runs on Core 1, locking the ISR there. When a Core 0 task uses that SPI bus, transactions appear to complete but the ISR services them from the wrong core. The cross-core IPC overhead silently corrupts transactions at the hardware level — no crash, no error code, just malformed data reaching the radio.

The Fix

Re-initialize SPI inside the Core 0 task function itself, so the ISR registers on Core 0:

```
SPI.end();                // Tear down Core 1's SPI registration
delay(2);                 // Allow cleanup
SPI.begin(SCK, MISO, MOSI, CSN); // Re-init on Core 0
delay(5);                 // Allow stabilization
```

Key takeaway: Always initialize SPI on the same core that will use it. If your radio task runs on Core 0, you MUST re-initialize SPI inside the task. Not before, not after. Inside it.

Implementation Highlights

Continuous Carrier with Frequency Hopping

Rather than the typical stop-reconfigure-transmit cycle for each channel hop (which creates dead air), HaleHound uses the NRF24L01's continuous carrier mode. The carrier is started once with startConstCarrier() and never stops — setChannel() simply steers the frequency while the carrier remains active. Zero ramp-up time. Zero dead air between hops.

Cross-Core Communication

All inter-core communication uses volatile variables only. This works safely because: single-word types (bool, int) are atomic on 32-bit Xtensa; every variable is unidirectional (one core writes, the other reads); display reads are best-effort (a stale frame is acceptable); and control flags use simple polling loops. A mutex here would defeat the entire purpose — Core 0's radio loop would block every time Core 1 reads display data.

Watchdog Handling

FreeRTOS requires periodic yields to feed the Task Watchdog Timer (TWDT). A tight radio loop starves the idle task. The fix: yield for 1 tick (~1ms) every 100 hops. At ~50µs per hop, this means 1ms of yield per ~5ms of radio work, giving a worst-case 83% duty cycle, with practical modes achieving ~99%.

Proof of Success

The radios work.

This architecture is not theoretical. It is not a proof-of-concept. It is not a simulation. It is deployed across thirteen production modules on physical hardware, verified with spectrum analyzer measurements, and running in the field right now.

Spectrum Analyzer Verification

The SPI ISR core-affinity discovery was identified and verified exclusively through spectrum analysis. Single-core baseline measurements showed clean, strong continuous carrier output at RF24_PA_MAX. Initial dual-core attempts showed weak, inconsistent carrier — visible only on the spectrum analyzer, invisible to serial output or software-side diagnostics. After applying the SPI re-initialization fix, dual-core output matched the single-core baseline: clean, full-power, continuous carrier. The spectrum analyzer doesn't lie.

Empirical Evidence

Evidence	Method	Result
Carrier signal quality	Spectrum analyzer comparison	Clean full-power CW on dual-core matches single-core baseline
Radio duty cycle	Timing analysis (hop count / sec)	~99% sustained — 2.23x improvement over 44% single-core
Jamming effectiveness	Target reconnection monitoring	Zero reconnection windows vs ~5.5/sec on single-core
FFT compute offload	Display frame rate measurement	~2 FPS single-core to 30+ FPS dual-core (15x improvement)
Touch responsiveness	Input latency measurement	Immediate response vs up to 100ms delay
Production stability	13 modules in continuous use	Zero crash, zero SPI corruption across all radio types
Hardware coverage	Multi-radio validation	WiFi, BLE, NRF24 2.4GHz, CC1101 Sub-GHz, DSP/FFT — all verified

Every single TX module injects +123% more packets per second compared to any single-core ESP32 RF tool on the market. Period.

Performance Results

Radio Duty Cycle Comparison

Metric	Single-Core	Dual-Core	Improvement
Radio active time / sec	444ms	990ms	+123%
Radio duty cycle	44.4%	~99%	2.23x
Max radio silence gap	~100ms	~1ms	99% smaller
SPI bus contention	Alternating (shared)	None (separate HW)	Eliminated
Hardware cost for equiv.	2x ESP32	1x ESP32	50% cheaper

Jamming Effectiveness

Metric	Single-Core	Dual-Core	Improvement
Max gap in jamming	100ms	1ms	100x smaller
BLE reconnection windows/sec	~5.5	~0	Eliminated
WiFi reassoc windows/sec	~5.5	~0	Eliminated
Carrier continuity	Interrupted	Continuous	Zero dead air

DSP/FFT & UI Performance

Metric	Single-Core	Dual-Core	Improvement
PacketMonitor FFT display	~2 FPS	30+ FPS	15x faster
SubGHz Replay FFT display	~2 FPS	30+ FPS	15x faster
Touch input latency	up to 100ms	Immediate	100ms eliminated
Animation smoothness	Drops frames during TX	Locked 30+ FPS	Consistent

Competitive Landscape

Platform	Duty Cycle	TX Rate	Jam Gap	HW Cost	Architecture
Marauder	44%	1x	100ms	1x ESP32	Single-core
Double Barrel Marauder	~99%	~2.2x	~1ms	2x ESP32	Two physical chips
RF-Clown	44%	1x	100ms	1x ESP32	Single-core
Ghost ESP	44%	1x	100ms	1x ESP32	Single-core

Platform	Duty Cycle	TX Rate	Jam Gap	HW Cost	Architecture
Flipper Zero	N/A	N/A	N/A	STM32	Single-core MCU
HaleHound-CYD (13 modules)	99%	2.23x	1ms	1x ESP32	Software dual-core

HaleHound-CYD matches the radio performance of dual-chip commercial boards at half the hardware cost. One chip. Software architecture. That's it.

Production Modules (13 Total)

WiFi Attack Modules

Module	Task Name	Core 0 Role
Beacon Spammer	"BSTx"	Continuous 802.11 beacon injection
Beacon Nuke	"NukeTx"	Multi-SSID beacon flooding at max rate
Deauther	"DTTx"	Deauth frame injection per target
Auth Flood	"AFTx"	Auth frame storm against target AP
PacketMonitor FFT	"PmFFT"	Real-time FFT spectrum computation from captures

BLE Attack Modules

Module	Task Name	Core 0 Role
BLE Spoofer	"SpamTx"	Raw ESP-IDF advertising with address rotation
BLE Beacon	"BcnTx"	Custom BLE advertisement broadcast
BLE Jammer	"BleJam"	NRF24 continuous carrier on BLE adv channels

NRF24 2.4GHz Attack Modules

Module	Task Name	Core 0 Role
WLAN Jammer	"WLANJam"	NRF24 carrier hopping across WiFi band
ProtoKill	"ProtoKill"	Multi-protocol 2.4GHz disruption with adaptive dwell

Sub-GHz CC1101 Modules

Module	Task Name	Core 0 Role
SubGHz Jammer	"SJam"	CC1101 wideband noise generation across bands
SubGHz Replay FFT	"SgFFT"	Real-time FFT visualization during signal capture
SubGHz Analyzer	"SAScan"	CC1101 sweep across 33 bands (300–928MHz)

Shared Architecture Pattern

Stack: 4096–8192 bytes depending on module complexity

Priority: 1 (above idle, below system tasks)

Core: 0 (dedicated radio/compute core)

Communication: volatile shared state with frameReady flag for lock-free sync

Watchdog: vTaskDelay(1) every N iterations to feed the idle task

Safe exit: Task sets taskDone = true + handle = NULL before vTaskDelete(NULL)

SPI ownership: Core 0 task calls SPI.end() SPI.begin() to re-register ISR, then SPI.end() on exit

Applying This to Your Project

1. Verify SPI bus layout — Confirm display and radio use separate HSPI/VSPI peripherals. Shared bus still works with a mutex but reduces duty cycle.

2. Pin a FreeRTOS task to Core 0 — Use xTaskCreatePinnedToCore() with core parameter 0, stack 4096–8192 bytes, priority 1.

3. Re-initialize SPI inside the task — MANDATORY. Call SPI.end() then SPI.begin() within the task function so the ISR registers on Core 0.

4. Use volatile variables for cross-core state — Keep it simple. Mutexes kill duty cycle. Single-word atomics are sufficient.

5. Yield to the watchdog — Insert vTaskDelay(1) every ~100 iterations to feed the TWDT idle task.

6. Clean up SPI on task exit — Call SPI.end() before vTaskDelete(NULL), then re-init on Core 1 from calling code.

7. Never touch the radio's SPI bus from Core 1 while the Core 0 task is running. Use a done-flag to gate re-initialization.

License & Legal

MIT License

Copyright (c) 2026 Jesse C. Hale (HaleHound-CYD Edition)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Attribution Requirements

For direct use: Retain the LICENSE file in any distribution. Maintain copyright notices in source files.

For modifications and forks: Include original copyright notice. Clearly indicate modifications made. Suggested format: "Based on HaleHound-CYD by Jesse C. Hale" with repository link.

For derivative works: You may create derivative works under the same MIT license. Original attribution chain must be preserved. You may add your own copyright for your modifications.

Contribution Terms

By submitting contributions to this repository (via pull requests, issues, or other means), you agree that: (1) your contributions are provided under the same MIT License; (2) you have the right to submit the contribution; (3) you grant the project maintainer rights to use, modify, and distribute your contribution as part of this project; (4) contributions become part of the collective work and are subject to this license.

Disclaimer & Responsible Use

AUTHORIZED USES: Authorized penetration testing (with explicit written permission), security research (studying wireless vulnerabilities and developing defensive measures in controlled environments), educational purposes, professional security assessments conducted by qualified professionals with proper scope agreements, and ham radio / RF experimentation in compliance with local regulations and licensing requirements.

PROHIBITED USES: Unauthorized access to wireless networks or systems; disruption of legitimate wireless communications; jamming or interfering with emergency services or aviation; any activity that violates local, state, federal, or international law; malicious attacks against individuals or organizations; use against systems without explicit written authorization.

REGULATORY NOTICE: Certain functions may be restricted or prohibited in your jurisdiction. RF transmission capabilities must comply with local regulations. Users are responsible for obtaining necessary licenses (e.g., HAM radio). Some features may violate FCC Part 15 or equivalent regulations. Unauthorized access to computer systems and wireless networks is a crime in most jurisdictions. Jamming or interfering with wireless communications may violate federal law (47 U.S.C. § 333).

The author(s) assume NO LIABILITY for any misuse, damage, or legal consequences arising from the use of this software or hardware. By using this software, you acknowledge that you understand the legal implications of wireless security testing, you will obtain proper authorization before any testing, you accept full responsibility for your actions, and you will comply with all applicable RF and wireless regulations.

Hardware Disclaimer

This firmware is designed for specific ESP32-based hardware configurations. Improper use may result in: damage to your ESP32 device or connected peripherals, RF interference with nearby devices, violation of radio frequency regulations, or voiding of hardware warranties. The authors provide no warranty for hardware compatibility or safety. Users assume all risk when flashing and operating this firmware.

Prior Art Statement

This dual-core RF architecture was independently developed by Jesse C. Hale of HaleHound for the HaleHound-CYD platform beginning in 2025. The SPI ISR core-affinity discovery — specifically that cross-core SPI transactions produce silently degraded RF output rather than errors, identifiable only through spectrum analysis — and the complete core-isolation pattern described in this document were developed through original research and iterative testing with spectrum analyzer verification.

At the time of publication (February 2026), no known ESP32-based RF security tool — open source or commercial — implements dedicated core pinning for radio operations with full SPI re-initialization on the radio core. Commercial products achieving comparable radio/display isolation do so by adding a second physical microcontroller, at greater hardware cost and complexity, to solve a problem that this architecture solves entirely in software.

Repository: github.com/JesseCHale/HaleHound-CYD